

Bugfix Log

Revision: 5 Last modified: 2026-06-09T20:00:00Z

Per CONST-MD-Bugfix-Documentation, every bug surfaced during implementation gets a permanent entry below: title, root cause, affected files, fix description, regression guard.

Entries are append-only; do not edit historical entries except to add clarification footnotes.

2026-04-27 "boba-jackett implementation (plan 2026-04-27-jackett-management-ui-and-system-db.md)

1. Master-key two-step write silent data-loss window

Severity: HIGH (silent credential loss possible on crash mid-bootstrap).

Root cause: Initial implementation of `bootstrap.EnsureMasterKey` generated `BOBA_MASTER_KEY` in memory, returned it to the caller, and *then* wrote `.env` in a separate step. If the process crashed, was killed, or the host lost power between those two operations, the caller would proceed to encrypt credentials with a key that disk had no record of "making them permanently undecipherable.

Affected files: - `qBittorrent-go/internal/bootstrap/bootstrap.go` - `qBittorrent-go/internal/envfile/write.go`

Fix: Collapse the generate + write into a single `envfile.Atomic` write (`tmpfile + fsync + rename`) before the function returns. Caller never sees a key that hasn't been durably persisted. Commit `c0f20bc`.

Regression guard: - `qBittorrent-go/internal/bootstrap/bootstrap_test.go::TestEnsureMasterKeyDoesNotDuplicateHeader` - `qBittorrent-go/internal/envfile/write_test.go` atomic-write test layer

2. SQLite database file mode 0644 (world-readable credentials)

Severity: CRITICAL (any local user could read encrypted credentials + ciphertext-only attack surface).

Root cause: `modernc.org/sqlite` creates database files with the process's default `umask`, which on the container yielded `0644` (world-readable). The Go service's `.env` was already `0600` via `envfile.Atomic`, but `boba.db` was leaking "a defense-in-depth breach since the rows are encrypted, but still a violation of the principle "the DB file should be readable only by the service owner".

Affected files: - `qBittorrent-go/internal/db/conn.go`

Fix: After the first `Open()` succeeds, `db.Open` calls `os.Chmod(path, 0o600)` (and the `-wal / -shm` siblings if present), collapsing the file mode to owner-only. Commit `8405167`.

Regression guard: - Layer 4 security challenge challenges/scripts/boba_db_file_perms_challenge.sh - qBitTorrent-go/internal/db/credential_leak_test.go

3. AutoconfigResult nil slices marshalled as JSON null

Severity: MEDIUM (silent client-breaking schema drift; UI rendered "undefined" badges instead of "0 items").

Root cause: Go's json.Marshal serializes a nil slice as null, not []. When Autoconfigure() ran on a fresh DB with no discovered credentials, the discovered_credentials, configured_now, and already_present fields all marshalled as null. The OpenAPI 3.1 schema declared them as array, so clients (including the Angular dashboard) treated null as a contract violation.

Affected files: - qBitTorrent-go/internal/jackett/autoconfig.go

Fix: Pre-allocate empty slices ([]string{}) at the top of Autoconfigure() so a "nothing to do" run still serializes as { ..., "discovered": [], "configured_now": [], ... }. Commit 6f3dbaf.

Regression guard: - Layer 6 contract test qBitTorrent-go/tests/contract/openapi_test.go (validates each named field is array per OpenAPI schema even on empty runs).

4. Catalog ReplaceAll empty-input wipe risk

Severity: HIGH (would permanently wipe the indexer catalog if Jackett returned an empty response; UI would show no indexers and fuzzy matcher would have nothing to match against).

Root cause: repos.IndexerCatalog.ReplaceAll(rows []Row) unconditionally DELETED existing rows then INSERTed the new ones. If rows was empty (Jackett momentarily unhealthy, transient 502, or catalog parse failure), the delete still happened, leaving an empty catalog table. Next refresh would have nothing to compare against.

Affected files: - qBitTorrent-go/internal/db/catalog.go (or equivalent repo file)

Fix: Refuse the operation early with errors.New("repos: ReplaceAll refusing empty replacement"). The caller (autoconfig orchestrator) treats this as a soft error, logs it, and leaves the previous catalog intact. Commit 8d71df1.

Regression guard: - qBitTorrent-go/internal/jackettapi/catalog_test.go::TestRefreshCatalogAllTemplatesFailReturns200WithErrors

5. TestSearchHandler_QueueFull pre-existing flake (NOT a boba-jackett bug)

Severity: LOW (test-only; flagged for follow-up).

Status: NOT FIXED in this plan. Confirmed unrelated to boba-jackett work.

Location: qBitTorrent-go/internal/api/api_test.go

Symptom: Occasional intermittent failure under load when the search queue fills before the test's wait returns.

Action: Logged here for traceability so future audits don't mis-attribute it. Open issue suggestion: stabilise by injecting a deterministic queue-full hook instead of racing against real goroutine scheduling.

2026-06-09 "Session 7 fixes (env_loader, AsyncMock, gamestorrents)

6. env_loader flaky test: KEY2 leak across test ordering

Severity: LOW (test-only flake; not a production bug).

Root cause: test_comment_lines_ignored in tests/unit/test_env_loader.py asserted os.environ.get("KEY2") is None after calling load_env_files() with a config file where KEY2=commented_out was commented out. However, load_env_files has a "first wins" policy if KEY2 was already present in os.environ from a prior test's load_env_files call, the function would not override it. Under pytest-randomly ordering, the test that sets KEY2 sometimes ran before the comment-line test, causing intermittent failure.

Affected files: - tests/unit/test_env_loader.py

Fix: Added explicit os.environ.pop("KEY1", None) and os.environ.pop("KEY2", None) at the START of the test (before calling load_env_files), ensuring a clean env state regardless of test execution order. The finally block was retained as a safety net.

Regression guard: - tests/unit/test_env_loader.py::test_comment_lines_ignored ran 2147/2147 twice consecutively under random ordering with zero failures.

7. AsyncMock warning in search deep-coverage tests

Severity: LOW (test warning; not a production bug).

Root cause: test_iptorrents_login_no_cookies in tests/unit/merge_service/test_search_deep_coverage.py used AsyncMock() for login_resp and mock_session. These objects are used as context managers (async with session.post(...) as resp), not as directly awaited coroutines. AsyncMock.__aenter__ returns a coroutine, but the test's mock setup didn't properly wire __aenter__/__aexit__, causing "coroutine was never awaited" RuntimeWarning.

Affected files: - tests/unit/merge_service/test_search_deep_coverage.py

Fix: Changed AsyncMock() to MagicMock() with explicit __aenter__ and __aexit__ stubs, matching the actual usage pattern (context manager, not coroutine).

Regression guard: - tests/unit/merge_service/
test_search_deep_coverage.py â€” 0 AsyncMock warnings from this test (was 3).

8. gamestorrents _parse_size B-substring bug (documented, not fixed)

Severity: MEDIUM (plugin returns 0 for all realistic file sizes).

Root cause: Same class of bug as BOB-013 (torrentkitty). The `multipliers` dict in `_parse_size` iterates in insertion order: `{"B": 1, "KB": 1024, ...}`. Since "B" is a substring of "KB", "MB", "GB", and "TB", the check `if unit in size_str` matches "B" first for all realistic sizes. Then `size_str.replace("B", "")` leaves a trailing unit character (e.g. "35.2 G") which `float()` cannot parse, returning 0.

Affected files: - plugins/gamestorrents.py (line 74-86)

Fix: NOT FIXED in this session. Documented via tests that assert actual behavior (tests suffixed `_b_substring_bug`). Fix requires reordering the dict keys to check longest units first (same approach as BOB-013).

Regression guard: - tests/unit/
test_plugin_gamestorrents.py::TestParseSize â€” 8 tests documenting the bug across GB/MB/KB/TB/commas/uppercase.

2026-06-09 â€” Session 8 parallel plugin tests + gamestorrents fix

9. gamestorrents _parse_size B-substring bug (fixed)

Severity: MEDIUM (plugin returned 0 for all realistic file sizes).

Root cause: Same class of bug as BOB-013 (torrentkitty). The `multipliers` dict in `_parse_size` iterated in insertion order: `{"B": 1, "KB": 1024, ...}`. Since "B" is a substring of "KB", "MB", "GB", and "TB", the check `if unit in size_str` matched "B" first for all realistic sizes. Then `size_str.replace("B", "")` left a trailing unit character (e.g. "35.2 G") which `float()` could not parse, returning 0.

Affected files: - plugins/gamestorrents.py (line 74-86)

Fix: Reordered dict keys to check longest units first: `{"TB": ..., "GB": ..., "MB": ..., "KB": ..., "B": 1}`. Same approach as BOB-013.

Regression guard: - tests/unit/
test_plugin_gamestorrents.py::TestParseSize â€” 8 tests all pass with correct byte values for GB/MB/KB/TB/B/comma/uppercase.

10. piratebay import os placed after use (documented, not fixed)

Severity: LOW (only affects non-magnet torrent file downloads, which are rare).

Root cause: piratebay.py:175 has `import os` placed after `os.fdopen` on line 172. When downloading a non-magnet torrent file, `os.fdopen` is called before `os` is imported, causing `UnboundLocalError`.

Affected files: - plugins/piratebay.py (line 172-175)

Fix: NOT FIXED in this session. Documented via test `test_torrent_file_download_known_bug_os_import_order`. Fix requires moving `import os` to the top of the `download_torrent` method or module level.

Regression guard: - tests/unit/test_plugin_piratebay.py::TestDownloadTorrent::test_torrent_file_download_

11. torlock search() does not catch exceptions from retrieve_url()

Severity: LOW (network errors crash the entire search loop instead of failing gracefully).

Root cause: torlock.py search() calls retrieve_url() without a try/except, so a network error propagates up and crashes the search loop.

Affected files: - plugins/torlock.py

Fix: NOT FIXED in this session. Documented via test `test_search_exception_propagates`. Fix requires wrapping `retrieve_url()` in try/except with graceful error handling.

Regression guard: - tests/unit/test_plugin_torlock.py::TestSearch::test_search_exception_propagates

2026-06-09 “ Session 8 wave 2: nyaa/kickass/anilibra/torrentgalaxy/yts

12. nyaa download_torrent missing import re (documented, not fixed)

Severity: MEDIUM (any nyaa.si URL passed to `download_torrent` raises `NameError`).

Root cause: plugins/nyaa.py:170 calls `re.search()` but `import re` is absent from the module. The `search()` method works because `re` is imported transitively via other modules, but `download_torrent` fails with `NameError: name 're' is not defined`.

Affected files: - plugins/nyaa.py (line 170)

Fix: NOT FIXED in this session. Documented via tests that assert the `NameError`. Fix requires adding `import re` at the top of the file.

Regression guard: - tests/unit/test_plugin_nyaa.py::TestDownloadTorrent “ 6 tests including the `NameError` documentation.

13. kickass comma-separated size not matched by regex (documented, not fixed)

Severity: LOW (sizes like 1 , 234 . 5 MB silently parse to 0).

Root cause: The HTMLParser regex `\d+\ . \d+` doesn't match comma-separated numbers like 1 , 234 . 5 MB. The comma is not handled in the regex or the replacement logic.

Affected files: - `plugins/kickass.py`

Fix: NOT FIXED in this session. Documented via test `test_comma_in_size_not_matched_by_regex`. Fix requires updating the regex to handle commas.

Regression guard: - `tests/unit/test_plugin_kickass.py::TestHTMLParserFeed::test_comma_in_size_not_matched`

14. kickass crash-prone patterns (fixed)

Severity: MEDIUM (empty responses from `retrieve_url` crash the plugin).

Root cause: Three methods in `kickass.py` called `retrieve_url()` without try/except or empty-response guards: `__retrieve_download_link()`, `download_torrent()`, and `search()`. When `retrieve_url` returned None or empty string, subsequent `re.search/re.sub` calls crashed with `TypeError`.

Affected files: - `plugins/kickass.py` (lines 61, 74, 94)

Fix: Added try/except + empty-response guards to all three methods. Each now handles empty/None responses gracefully (returns `None`, prints fallback URL, or breaks the loop respectively).

Regression guard: - `tests/unit/test_plugin_kickass_guards.py` 13 tests proving empty response, None response, `ConnectionError`, `TimeoutError`, and malformed HTML don't crash.

2026-06-09 "Waves 5-8: B-substring epidemic across 9 community plugins

15. Systemic `_parse_size` B-substring bug (9 instances found, all fixed)

Severity: MEDIUM (all 9 plugins returned 0 for KB/MB/GB/TB "silently broken size reporting").

Root cause: The `_parse_size` method in each plugin uses a `multipliers` dict iterated in insertion order. With keys ordered `{"B": 1, "KB": 1024, ...}`, the check "B" in `size_str` matches the "B" substring inside "KB", "MB", "GB", and "TB". Then `size_str.replace("B", "")` leaves a trailing character (e.g. "1.5 G") which `float()` cannot parse, returning 0.

Affected plugins (9 total): | Plugin | Status | | gamestorrents.py | Fixed (BOB-024) | | megapeer.py | Fixed (BOB-042) | | one337x.py | Fixed (BOB-043) | | extratorrent.py | Fixed (BOB-044) | | torrentfunk.py | Fixed (BOB-045) | | therarbg.py | Fixed (BOB-047) | | pctorrent.py | Fixed (BOB-052, by subagent) | | bitru.py | Fixed (BOB-054) | | xfsu.py | Fixed (BOB-057) | | yihua.py | Fixed (BOB-058) |

Fix: Reorder dict keys longest-unit-first: {"TB": ..., "GB": ..., "MB": ..., "KB": ..., "B": 1}. Same approach as BOB-013 (torrentkitty).

Global regression guard: 10 dedicated test files, each with explicit `_parse_size` assertions for all 5 units (B/KB/MB/GB/TB).

16. Systematic import re omissions (3 instances found, all fixed)

Root cause: Three plugins called `re.search()` or `re.compile()` without importing `re` at module level. The `search()` method worked via transitive imports from helper modules, but `download_torrent()` failed with `NameError`.

Affected plugins: | Plugin | Status | | nyaa.py | Fixed (BOB-035) | | audiobookbay.py | Fixed (BOB-042) | | (torlock `search()` also lacks exception handling â€” documented BOB-029) |

Fix: Added `import re` at module level.

17. bt4g infinite loop (test-only, fixed)

Severity: LOW (test hang, not production).

Root cause: `search()` has a `while True` loop that breaks when `parser.noTorrents` is `True`. Tests that used `return_value=MATCHING_HTML` (not `side_effect`) caused every page to match, triggering infinite loop.

Fix: Changed to `side_effect=[MATCHING_HTML, EMPTY_HTML]` so page 2 triggers the `noTorrents` break condition.